

VERIFIED ACCESSIBILITY REMEDIATION WITH AGENTS

micro1 Agentic Workflows Hackathon, solution report Submitted by: Muneeb Wasi Khan · August 2026 ·
Repository: [a11y-agent/](#)

An agentic workflow that fixes web accessibility defects and proves, in a real browser, that its fixes hold where automated linters cannot see: keyboard reachability, activation behavior, focus indicators, dialog focus management, and visual integrity.

EXECUTIVE SUMMARY

We compared a one-shot model call against the same model equipped with verification tools and the ability to iterate, on 12 seeded fixture pages, scored by a three-gate browser harness. On [claude-haiku-4-5](#), three sweeps per arm:

METRIC	ONE-SHOT BASELINE	AGENT WITH VERIFICATION TOOLS
Pass rate per sweep, all three gates	4/12, 7/12, 5/12	11/12, 12/12, 11/12
Worst sweep, the reliability floor	4/12	11/12
Cases passing every sweep	4 of 12	11 of 12
Cost per 12-case sweep, client estimate	\$0.09	\$0.70
Machine time per case, human minutes zero	7.4s	110s

The gains concentrate exactly where scanners are blind: the five cases the baseline never passes in any sweep (contrast, form errors, landmarks, a custom dropdown, a modal dialog) flip to passing every sweep with tools, with one honest exception documented below. Every number is independently reproducible from the committed archive with no API key: [npm run verify](#).

WHO HAS THE PROBLEM

Frontend teams shipping under WCAG 2.1 AA obligations. The European Accessibility Act has applied to most consumer-facing digital products since June 2025, and accessibility lawsuits keep climbing. An audit lands on a developer's desk as dozens of violations across components.

The bottleneck is verification, not typing. Every fix needs a re-scan, a visual check and a keyboard pass. The tempting fixes are often fake: ARIA sprinkled until the scanner goes quiet can leave a page less

usable for the people the fix was meant to serve. Scanners like axe-core catch only a minority of real defects and none of the behavioral ones: a mouse-only dropdown, a dialog that never traps focus, an `outline: none` that erases every focus indicator.

WHAT WE BUILT

```
fixtures (12 seeded pages)          the exam, spec seen only by the grader
|
v
baseline arm          agent arm      same model, same contract, same files
one API call          SDK agent with audit_page, check_keyboard,
no tools              preview_page tools, iterates until clean
    \                /
    v                v
scoring harness (headless Chromium)
gate 1  axe: zero violations, WCAG 2.1 A + AA + best practice
gate 2  keyboard: real Tab presses, Enter and Space activation
        via an instrumentation hook, focus indicator detection,
        behavioral probes for the modal trap and dropdown arrows
gate 3  layout: DOM geometry compared to the original, nothing
        lost, collapsed or overflowing, design intact
|
v
results.json / REPORT.md -> runs-archive/ (committed)
                        |
                        v
                        npm run verify (judges, no API key)
```

A case passes only when all three gates hold. That conjunction is the anti-gaming design: a solution cannot win the axe gate by deleting elements, and cannot win by ARIA band-aids that a real key press exposes.

Sandbox, so the numbers can be trusted. The agent works in an isolated copy: edits auto-approved only inside it, read denials on the harness, fixtures, runners and archive, no shell and no network tools, no permission bypass of any kind. Every denied attempt is counted into the results. Its tools return raw observations, never thresholds or expected-element lists.

Purposeful capabilities, one lever at a time. The runner exposes variants (`v1-notools` through `v5-memory`) so each changelog row attributes improvement to a single design choice: iteration alone, plus the axe audit tool, plus keyboard observation and screenshots, plus a fix-patterns skill, plus cross-case memory.

EXPERIMENT DESIGN, INCLUDING THE PART THAT FAILED

Fairness is structural: both arms receive the identical task contract and fixture files, and the baseline uses structured outputs so it can never lose points to response formatting. The deliberate delta is tools

plus iteration. That delta is the experiment.

Benchmark v1 taught us the eval was wrong. Our first contract itemized every requirement, the focus trap recipe, the ARIA listbox pattern, click parity. On `claude-sonnet-5`, the one-shot baseline scored 12/12 and so did the no-tools agent at nearly triple the cost. A contract that contains the rubric measures spec-following, not capability. We recorded it as our removed experiment, cut the contract down to what a real ticket says (meet WCAG 2.1 AA from the keyboard without changing the design), and re-ran: sonnet still one-shots small textbook pages (11/12 with one slip), so headroom had to come from model strength, not contract wording.

The final experiment: the ladder on `claude-haiku-4-5`, where raw model knowledge stops carrying everything, three sweeps per arm to measure reliability rather than luck.

RESULTS IN FULL

Per-case passes across the three sweeps of each arm:

CASE	UNTREATED AXE VIOLATIONS	BASELINE	AGENT
01-missing-labels	4	3/3	3/3
02-low-contrast	9	0/3	3/3
03-div-buttons	0	3/3	3/3
04-missing-alt	4	3/3	3/3
05-heading-order	2	1/3	3/3
06-focus-indicators	0	3/3	3/3
07-modal	0	0/3	1/3 *
08-form-errors	3	0/3	3/3
09-custom-dropdown	0	0/3	3/3
10-landmarks	5	0/3	3/3
11-lang-title	2	1/3	3/3
12-tabindex-order	5	2/3	3/3

* The modal's two agent misses are one genuine failure (focus did not return to the opener after close, the subtlest behavior in the set) and one sweep voided by API-side request stalls, recorded as `failReason: timeout` with zero cost in the archived results. We report it as a miss rather than excluding it.

Note the rows where untreated axe violations are zero (03, 06, 07, 09): those defects are invisible to the scanner and only the keyboard gate measures them. They are also where the baseline-to-agent gap lives.

The challenging case (09, custom dropdown) was designed for exactly this: a styled div dropdown that works perfectly with a mouse and scores a clean axe scan, yet cannot be reached or operated from the keyboard. The baseline never fixes it. The agent fixes it every sweep, because its keyboard tool shows it the truth (a walkthrough is in Appendix A).

WHICH CAPABILITY CARRIES THE GAIN

Ablations on the same model, adding one capability per rung (intermediate rungs are single sweeps, the endpoints carry multi-sweep evidence):

RUNG	CAPABILITY ADDED	PASS RATE	WHAT STILL FAILS
baseline, x3	none	4/12, 7/12, 5/12	a bit of everything
v1, x1	iteration	9/12	dropdown, plus two axe-visible cases it cannot self-check
v2, x1	+ axe audit tool	10/12	only the modal and dropdown, both axe-clean, both keyboard failures
v3, x3	+ keyboard and screenshot tools	11/12, 12/12, 11/12	one modal focus-return flake
v4, x1	+ WCAG fix-patterns skill	12/12	nothing, first perfect sweep
v5, x1	+ cross-case memory	11/12	one dropdown flake, memory neutral at one sample

The dissociation is the finding: the scanner tool fixes what the scanner sees (v2 recovers the axe-visible cases and moves nothing behavioral), the keyboard tool fixes what users feel (v3 closes the two cases axe scored as perfect), and the skill steadies the subtlest behavior, the modal's focus return.

THE IMPROVEMENT JOURNEY

The full evidence-linked log is in [changelog.md](#), one row per experiment with run directories a judge can re-score. The arc in one paragraph: untreated fixtures established a 0/12 floor with seeded defects verified to register. Benchmark v1 saturated (both arms 12/12 on sonnet), which exposed the spec-leaking contract, our removed experiment. Benchmark v2 with the realistic contract still saturated on sonnet (11 to 12 out of 12), proving pass-at-first-attempt on textbook pages measures little. The haiku reliability ladder then produced the headline: a 4/12 floor one-shot against an 11/12 floor with verification tools, with five never-passing cases flipping to always-passing. Along the way, two sweeps were voided by API weather, and the runner grew a 20 minute fuse, an automatic fresh-copy retry per case, and strict MCP isolation, reliability engineering the eval needed as much as the agent did.

MAIN FAILURE MODE AND HOT TAKE

Failure mode. Unverified generation produces plausible-looking accessibility: markup that reads correctly and fails a real key press. Five defect classes scored zero for three one-shot sweeps. Even with tools, the last thing to fall was the modal's focus return, behavior three steps deep into an interaction.

Hot take. An agent optimizes whatever its verifier checks, and so does a benchmark. Our first contract contained the rubric and every arm saturated. Pass at first attempt measures luck, worst-of-N against behavioral gates measures deployable quality. Put the engineering effort into the verifier, it is the only part of the system the model cannot fake.

REPRODUCING EVERY NUMBER, NO API KEY

```
npm ci
npm run setup
npm run verify
```

`verify` re-serves every archived working copy, re-runs the axe scan, the keyboard checks and the layout comparison in a real browser on your machine, and diffs the recomputed gates against the committed results. Expected output: MATCH on every case (180 comparisons across 15 archived runs), then the regenerated comparison table. Roughly fifteen minutes, zero cost, nothing leaves the machine. Regenerating runs from scratch needs a key and is documented in `REPRODUCE.md`, about \$0.09 per baseline sweep and \$0.70 per agent sweep on haiku.

Ground rules compliance: everything here was built during the competition, all data is synthetic, credentials stay outside the repository, the agent is sandboxed with denials logged, and the intended workflow keeps a developer reviewing the produced diff before anything ships.

APPENDIX A: ONE AGENT TRAJECTORY, ABRIDGED

Case 09, custom dropdown, `claude-haiku-4-5` with verification tools. Faithful excerpts from `runs-archive/agent-haiku-2/09-custom-dropdown/trajectory.md`:

Turn 5: tool call `Read {"file_path": "/index.html"}` -> error: Read is not permitted outside the case working copy (the sandbox denies an absolute-path read, the agent recovers with a relative path, and the denial is counted in the results)

Turn 10: tool call `audit_page` -> "axe found 0 violation nodes... Automated scanning misses most keyboard and focus behavior, verify those separately."

Turn 13: tool call `check_keyboard` -> "NOT reachable with Tab: dd-toggle, dd-option-1, dd-option-2, dd-option-3. activation: dd-toggle + enter -> NO effect..."

Turn 18: "Now I understand the issues... The custom dropdown widget needs keyboard support. Let me fix this by making dd-toggle focusable, adding keyboard event handlers, adding appropriate ARIA roles."

Turn 50: after edits, `check_keyboard` again -> toggle now activates with Enter and Space, options still unreachable, so the agent keeps working instead of declaring victory.

Turn 53: final `audit_page` -> clean, followed by a change summary.

Result: success in 20 turns, 143 seconds, \$0.12, 1 permission denial recorded.

The scanner said the page was fine. The keyboard tool said it was not. The agent believed the tool, caught its own incomplete fix mid-loop, and finished the job. That is the entire thesis in one trajectory.

APPENDIX B: DELIVERABLES MAP

BRIEF DELIVERABLE	WHERE IT LIVES
Complete solution code and improvement changelog	repository root, <code>README.md</code> , <code>changelog.md</code>
Reproduction guide	<code>REPRODUCE.md</code> , key-free tier first
Solution video	submitted alongside the repository
Agent trajectories	<code>trajectory.jsonl</code> and readable <code>trajectory.md</code> per case in every archived agent run under <code>runs-archive/</code>
Measured results	<code>runs-archive/REPORT.md</code> , regenerated by <code>npm run verify</code>